

HO CHI MINH UNIVERSITY OF TECHNOLOGY, HCMC

FACULTY OF COMPUTER SCIENCE & ENGINEERING



Software Requirements Specification for Guide to Southern Woman's Museum

Prepared by: Tran Tuan Minh Khoa (2252365)

Nguyen Huu Tri (2252842)

Tran Phu Duc (2210814)

Ngo Phan Khai Tu (2153951)

Supervised under: Phan Tran Minh Khue

May 14, 2026

Contents

1	Introduction	5
1.1	Purpose	5
1.2	Product Scope	5
1.3	Definitions, Acronyms, and Abbreviations	6
1.4	References	7
2	Overall Description	8
2.1	Product Perspective	8
2.2	Product Functions	8
2.3	User Classes	9
2.4	Operating Environment	9
2.5	Design and Implementation Constraints	9
2.6	Assumptions and Dependencies	10
3	Functional Requirements	11
3.1	Use Case Diagram	11
3.2	Use Case 1: Authenticate User	11
3.3	Use Case 2: Browse Museum Map	13
3.4	Use Case 3: Explore Room Artifacts	14
3.5	Use Case 4: Browse News and Events	15
3.6	Use Case 5: Manage User Profile	17
4	Non-functional Requirements	19
4.1	Performance	19
4.2	Security	19
4.3	Usability	19
4.4	Maintainability	20
4.5	Scalability	20
5	External Interface Requirements	21
5.1	User Interface	21
5.2	Software Interfaces	22

5.3	Communication Interfaces	22
6	System Architecture	23
6.1	Architectural Overview	23
6.2	Backend Module Structure	24
6.3	Flutter Screen Hierarchy	24
6.4	JWT Authentication Data Flow	25
7	API Endpoint Summary	26
8	Database Schema Summary	28

List of Tables

3.1	Use case – Authenticate User	11
3.2	Use case – Browse Museum Map	13
3.3	Use case – Explore Room Artifacts	14
3.4	Use case – Browse News and Events	15
3.5	Use case – Manage User Profile	17

List of Figures

3.1	System Use Case	11
6.1	Three-layer system architecture	23
6.2	System Use Case	24
6.3	System Use Case	24
6.4	JWT authentication data flow	25

Chapter 1

Introduction

1.1 Purpose

In today’s digital era, many aspects of daily life are transitioning from traditional methods to mobile and computer-based solutions. However, museum tour guidance in Vietnam remains largely under-digitalized. Many museums still rely on paper maps or human tour guides, which can limit visitors’ flexibility and personal exploration experience.

A notable example is the Southern Women’s Museum (Bao Tang Phu Nu Nam Bo), located at 202 Võ Th Sáu Street, District 3, Ho Chi Minh City. Established on 29 April 1985, the museum was initiated in 1983 by Mrs. Nguyn Th Thp (former Chairwoman of the Vietnam Women’s Union) together with 13 veteran female cadres. It showcases the bravery, resilience, and craftsmanship of Southern Vietnamese women throughout history through its rich and meaningful exhibitions. Although the museum provides an official website filled with extensive information, the platform can feel overwhelming and difficult to navigate on handheld devices. More importantly, it lacks the focused functionality of an actual tour guide, leaving many visitors unsure of where to begin their journey upon arriving at the museum — an issue we personally experienced as well.

To address these challenges, we developed *Guide to Southern Women’s Museum*, a digital “pocket” tour guide designed to enhance the museum visit experience. The application provides navigation and orientation support, digitised artifact information, and a more interactive and engaging exploration experience for visitors.

1.2 Product Scope

Guide to Southern Women’s Museum is a cross-platform mobile application built with Flutter (Dart) on the frontend and NestJS (TypeScript) on the backend. The system is composed of two main tiers:

- **Mobile Application (Client):** A Flutter-based app for Android/iOS that al-

allows visitors to log in, browse museum rooms, view artifact details with their positions on a room map, explore current events, and manage their personal profile and settings.

- **REST API Server (Backend):** A NestJS server backed by a PostgreSQL database hosted on Supabase, exposing RESTful endpoints for authentication, room management, artifact management, event management, and user profile management. API documentation is auto-generated via Swagger/OpenAPI at `/api/docs`.

Key product features include:

- User registration and JWT-based authentication.
- Interactive 2D museum floor map (Ground Floor and 1st Floor) drawn with Flutter's CustomPainter.
- Room-level artifact explorer with normalised coordinate positioning on a room map panel.
- Bilingual support (Vietnamese and English) for room names and artifact descriptions.
- Museum event feed with status tracking (active, inactive, ended).
- User profile management with language preference, notification toggle, and dark/light theme.
- Admin-only endpoints for managing rooms, artifacts, and events.

The product is developed for the course *Mobile Application Development (CO3043)*, Vietnam National University Ho Chi Minh City — Ho Chi Minh City University of Technology, Faculty of Computer Science and Engineering.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
JWT	JSON Web Token – a compact, stateless authentication token signed with a secret key.
REST API	Representational State Transfer – an HTTP-based communication style between client and server.
ORM	Object-Relational Mapping – technique mapping code objects to database tables (TypeORM used here).

Term	Definition
UUID	Universally Unique Identifier – primary key format for all database entities.
DTO	Data Transfer Object – a class used to validate and transfer data between architectural layers in NestJS.
Artifact	A digitized museum exhibit item with description, image, and normalized position coordinates.
Room / Gallery	A physical gallery space identified by a unique code (e.g., R1.1, R3).
Normalized Coords	A pair $(positionX, positionY) \in [0.0, 1.0]$ representing a relative artifact position within a room map frame.
Supabase	A backend-as-a-service platform providing managed PostgreSQL.
Provider	A Flutter state-management pattern using <code>ChangeNotifier</code> to propagate state to the UI.
SharedPreferences	A lightweight key-value local store used to persist the JWT token across sessions.
AdminGuard	NestJS middleware restricting endpoint access to users with <code>isAdmin = true</code> in their JWT payload.
Glass-morphism	A UI design style using <code>BackdropFilter</code> + blur to create a frosted-glass navigation bar.

1.4 References

- Flutter Documentation: <https://docs.flutter.dev/>
- NestJS Documentation: <https://docs.nestjs.com/>
- TypeORM Documentation: <https://typeorm.io/>
- Supabase Documentation: <https://supabase.com/docs>
- Swagger / OpenAPI Specification: <https://swagger.io/specification/>
- JWT Standard (RFC 7519): <https://datatracker.ietf.org/doc/html/rfc7519>

Chapter 2

Overall Description

2.1 Product Perspective

Guide to Southern Women’s Museum consists of two independently deployable components: a Flutter mobile application (frontend) and a NestJS API server (backend).

The backend is written in TypeScript and follows the NestJS module architecture. Each module encapsulates a single business domain – Auth, Room, Artifact, Event, and UserProfile – with its own controller, service, entity, and DTOs. Data is persisted in PostgreSQL on Supabase via TypeORM with an SSL connection. The full API surface is auto-documented through Swagger UI at `/api/docs`.

The Flutter frontend uses the Provider pattern for global state. All HTTP communication is centralized in `ApiService`, which appends the `Authorization: Bearer {token}` header to every authenticated request. The JWT issued at login is saved in `SharedPreferences` so the session persists across application restarts.

The system supports two permission levels. *Visitors* (regular users) may browse all museum content and manage their own profile. *Administrators* additionally manage event content and reposition artifacts on room maps. Access is enforced at both the API layer (`AdminGuard`) and the Flutter UI layer.

2.2 Product Functions

The application delivers five major feature groups:

- (a) **Authentication and Account Management.** Users register with name, email, phone, and password (bcrypt-hashed). A successful login returns a signed JWT that grants access to all protected features.
- (b) **Interactive Museum Map.** The Map screen renders a two-floor plan (Ground Floor and 1st Floor) with Flutter’s `CustomPainter`. Rooms appear as tappable diamond shapes; a staircase connector strip separates the two floors.

- (c) **Room and Artifact Exploration.** Selecting a room loads its metadata and all artifacts from the API. Each artifact is plotted as a numbered dot on an interactive room map using its normalized coordinates. Adjacent rooms can be reached via inline navigation buttons.
- (d) **News and Events.** The Home screen fetches and displays the five most recent museum events, each showing a thumbnail, headline, and date.
- (e) **Personal Profile Management.** Users view and edit personal information, toggle dark mode, switch display language (English/Vietnamese), manage notification preferences, and sign out securely.

2.3 User Classes

Visitor (Regular User). The primary audience – museum guests who use the application during their visit. Visitors access all read-oriented features and manage their own profile after completing email-and-password authentication.

Administrator (Admin). A privileged user identified by `isAdmin = true` in the database. In addition to all visitor capabilities, administrators manage event content (create, update, delete) and reposition artifacts on room maps. Both `JwtAuthGuard` and `AdminGuard` protect these endpoints.

2.4 Operating Environment

The Flutter application targets Android and iOS. The NestJS backend runs on any Node.js-compatible environment with a Supabase PostgreSQL connection. All client-server communication uses HTTPS. Environment-specific configuration (API base URL, server port, CORS origins) is managed through `.env` files.

2.5 Design and Implementation Constraints

- The backend requires `DATABASE_URL` to be set before startup; its absence causes a fatal initialization error.
- Supabase TLS certificates require `ssl.rejectUnauthorized = false` in TypeORM connection options.
- The pagination parameter `limit` is capped at 100 items per page.
- Artifact coordinates must be in $[0.0, 1.0]$; the Flutter client clamps values to $[0.06, 0.94] \times$

[0.08, 0.92] to prevent overflow at map edges.

- TypeORM `synchronize` must be `false` in production to prevent unintended schema alterations.

2.6 Assumptions and Dependencies

The system assumes users have a stable Internet connection. If Supabase becomes unavailable, all API-dependent features will fail. The Flutter client assumes the backend is reachable at the URL specified by `API_URL`. Because no refresh-token mechanism is currently implemented, users must log in manually once their JWT expires. The museum floor plan is rendered client-side and requires no network access.

Chapter 3

Functional Requirements

3.1 Use Case Diagram

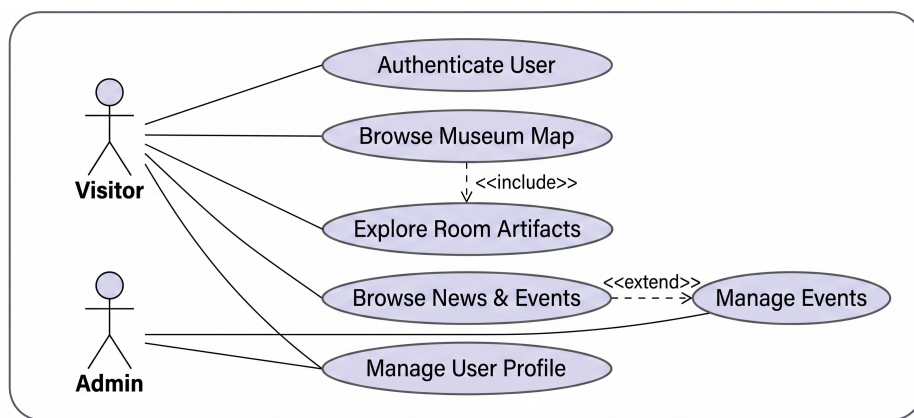


Figure 3.1: System Use Case

3.2 Use Case 1: Authenticate User

Table 3.1: Use case – Authenticate User

Field	Description
Use-case name	Authenticate User
Actors	Visitor
Description	A new or returning user registers or signs in to obtain a JWT access token. On success, the token is stored locally and the user enters the main application via a mode-selection dialog.
Trigger	The application launches and <code>AuthService</code> detects no valid token in <code>SharedPreferences</code> , redirecting to <code>LoginScreen</code> .

Continued...

Field	Description
Preconditions	The application is installed on a mobile device. The backend API is reachable. No prior authentication is required.
Postconditions	JWT token is stored in <code>SharedPreferences</code> . <code>AuthService.isAuthenticated</code> is <code>true</code> . <code>currentUser</code> holds <code>firstName</code> , <code>lastName</code> , <code>email</code> , and <code>phone</code> . The user is on <code>LayoutScreen</code> .
Normal flow (Sign In)	1. User enters email and password in <code>LoginScreen</code>. 2. App calls <code>POST /api/auth/login</code> with <code>{email, password}</code>. 3. Backend finds the user and verifies the password with <code>bcrypt.compare()</code>. 4. Backend creates JWT payload <code>{email, sub, isAdmin}</code>, signs it, and returns <code>{access_token}</code>. 5. Frontend stores the token and calls <code>apiService.setAuthToken()</code>. 6. App calls <code>GET /api/profile/me</code> to load user details. 7. <code>ModeSelectionScreen</code> slides up as a bottom-sheet dialog. 8. User selects a visit mode and proceeds to <code>LayoutScreen</code>.
Alternative flow (Sign Up)	1. User navigates to <code>SignupScreen</code>. 2. User enters <code>firstName</code>, <code>lastName</code>, <code>email</code>, <code>phone</code>, and <code>password</code>. 3. App calls <code>POST /api/auth/register</code>. 4. Backend checks for duplicate email (throws <code>409 Conflict</code> if found), then hashes the password with <code>bcrypt.hash(password, 10)</code> and saves the new user. 5. Backend returns <code>{message: "User registered successfully"}</code>. 6. Frontend redirects to <code>LoginScreen</code>.
Exception flow	401 Unauthorized: incorrect credentials – <code>AuthService.error</code> is set and displayed to the user. 409 Conflict: email already registered – the user is prompted to sign in or use a different email.

3.3 Use Case 2: Browse Museum Map

Table 3.2: Use case – Browse Museum Map

Field	Description
Use-case name	Browse Museum Map
Actors	Visitor (authenticated)
Description	The user views an interactive two-floor plan of the museum and taps a room shape to navigate into that gallery.
Trigger	The user selects the Map tab in the bottom navigation bar.
Preconditions	The user is authenticated and on <code>LayoutScreen</code> .
Postconditions	<code>LayoutScreen</code> updates <code>_selectedRoomCode</code> and <code>_currentRoute</code> to display the chosen <code>RoomScreen</code> .
Normal flow	1. <code>MapScreen</code> renders the floor plan via <code>CustomPainter</code>: 1st Floor: rooms R1.1, R1.2, R2.1, R2.2 as interactive diamond shapes with elevator pins. Ground Floor: room R3 and InfoDesk area, with elevator and restroom pins. 2. A <i>STAIRCASE CONNECTION</i> divider strip separates the two floors. 3. User taps a room diamond (e.g., R1.1). 4. <code>onRoomTap(code, floorLabel)</code> callback fires in <code>LayoutScreen</code>. 5. <code>LayoutScreen</code> sets <code>_selectedRoomCode = "R1.1"</code> and <code>_currentRoute = AppRouter.room</code>. 6. <code>RoomScreen</code> renders in the same scaffold without pushing a new route.
Alternative flow	The user browses the map without tapping any room; the map stays visible and no navigation occurs.
Exception flow	If map assets fail to render, a fallback background color is displayed while room labels and tap targets remain functional.

3.4 Use Case 3: Explore Room Artifacts

Table 3.3: Use case – Explore Room Artifacts

Field	Description
Use-case name	Explore Room Artifacts
Actors	Visitor (authenticated)
Description	After selecting a room, the user loads all of its artifacts and views their positions on an interactive room map alongside descriptive metadata.
Trigger	RoomScreen is mounted with a valid roomCode parameter provided by LayoutScreen.
Preconditions	A valid roomCode is passed from LayoutScreen. The user is authenticated and a network connection is available.
Postconditions	Room metadata and a list of up to 100 artifacts are displayed. Each artifact is plotted at its normalized coordinates on the room map.

Continued...

Field	Description
Normal flow	1. <code>RoomScreen.initState()</code> triggers <code>_loadRoom()</code>, which issues two sequential API calls: (a) <code>GET /api/rooms?code={roomCode}</code> – retrieves room id, name, nameEn, description, descriptionEn. (b) <code>POST /api/artifacts/all</code> with body <code>{page:1, limit:100, filters:{roomId}}</code> – retrieves artifacts sorted by <code>orderNo</code> ascending. 2. <code>RoomHeader</code> shows the English room name (fallback: Vietnamese), the floor label, and a back button. 3. Room description (English preferred) appears beneath the header. 4. <code>RoomMapPanel</code> plots each artifact as a numbered circle at coordinates (<code>width*positionX</code>, <code>height*positionY</code>), clamped to $[0.06, 0.94] \times [0.08, 0.92]$. 5. If adjacent rooms exist (<code>R1.1↔R1.2</code> or <code>R2.1↔R2.2</code>), a directional <i>Move to [code]</i> button appears inside the map panel. 6. A scrollable list of <code>ArtifactCard</code> widgets shows each artifact's thumbnail, name, and historical date. 7. If no artifacts exist, the message “<i>No artifacts found for this room.</i>” is displayed.
Alternative flow	The user taps <i>Move to [roomCode]</i> ; <code>LayoutScreen</code> updates <code>_selectedRoomCode</code> and rebuilds <code>RoomScreen</code> for the adjacent room.
Exception flow	If either API request fails, an <code>_ErrorState</code> widget is shown with a <i>Retry</i> button that re-triggers <code>_loadRoom()</code> .

3.5 Use Case 4: Browse News and Events

Table 3.4: Use case – Browse News and Events

Field	Description
Use-case name	Browse News and Events
Actors	Visitor (authenticated); Admin (for write operations)

Continued...

Field	Description
Description	Visitors read the latest news published by the museum. Administrators additionally create, update, and delete event records via protected endpoints.
Trigger	<code>HomeScreen.didChangeDependencies()</code> is called and the internal <code>_loaded</code> flag is <code>false</code> .
Preconditions	The user is authenticated and a network connection is available.
Postconditions	Up to five recent events are displayed on the Home screen. Admin write operations take effect immediately on the server.
Normal flow (Visitor)	1. <code>HomeScreen</code> calls <code>_loadEvents()</code> and sets <code>_loaded = true</code> to prevent duplicate fetches. 2. <code>ApiService.getEvents()</code> issues <code>GET /api/events</code> and parses the response into a <code>List<Event></code>. 3. A <code>CircularProgressIndicator</code> is shown while loading. 4. On success, up to five <code>NewsItem</code> widgets are rendered, each showing a thumbnail, truncated headline (max 2 lines), museum name, and event date. 5. If the list is empty, the message “<i>No news available</i>” is shown.
Alternative flow (Admin)	Create: <code>POST /api/events</code> with event payload. Update: <code>PATCH /api/events/:id</code> with partial payload. Delete: <code>DELETE /api/events/:id</code>. All write endpoints require a valid JWT with <code>isAdmin = true</code>, enforced by both <code>JwtAuthGuard</code> and <code>AdminGuard</code>.
Exception flow	On network failure, an error message from <code>ApiService</code> is surfaced in the UI. The user may navigate away and return to trigger a fresh fetch.

3.6 Use Case 5: Manage User Profile

Table 3.5: Use case – Manage User Profile

Field	Description
Use-case name	Manage User Profile
Actors	Visitor (authenticated)
Description	The user views and edits personal information, toggles application preferences (dark mode, language, notifications), accesses help information, and signs out securely.
Trigger	The user selects the Profile tab in the bottom navigation bar.
Preconditions	The user is authenticated and a valid JWT exists in SharedPreferences .
Postconditions	Updated profile data is synchronized with the server. UI preference changes take effect immediately without an application restart.
Normal flow (View)	1. <code>ProfileScreen.initState()</code> calls <code>_loadUserData()</code>. 2. If <code>currentUser</code> is absent, <code>GET /api/profile/me</code> is called to populate it. 3. An avatar circle shows the user's initials (<code>firstName[0]+lastName[0]</code>) on a primary-color background. 4. The screen displays full name, email, and a “<i>Museum member</i>” badge. 5. Personal Information section lists <code>firstName</code>, <code>lastName</code>, and <code>email</code> in read-only mode. 6. Settings section provides toggles for notifications, language (EN/VI), and dark mode, plus links to Privacy Policy and Help & Support.

Continued...

Field	Description
Alternative flow (Edit)	1. User taps <i>Edit</i>; <code>TextFormField</code> widgets appear. 2. User modifies fields and taps <i>Save changes</i>. 3. Form validates email format and non-empty fields. 4. App calls <code>PATCH /api/profile/update</code> with <code>{first_name, last_name, email, phone}</code>. 5. On success: green <code>SnackBar</code> “Profile updated successfully”. 6. On failure: red <code>SnackBar</code> with the error description.
Alternative flow (Sign Out)	1. User taps the <i>Sign Out</i> button. 2. A confirmation dialog appears (<i>Cancel / Sign Out</i>). 3. On confirmation: <code>AuthService.logout()</code> clears the JWT from <code>SharedPreferences</code>, nullifies <code>_accessToken</code> and <code>_currentUser</code>, and calls <code>notifyListeners()</code>. 4. <code>Navigator.pushNamedAndRemoveUntil('/login')</code> clears the back stack.
Alternative flow (Dark Mode)	User taps the Dark Mode toggle; <code>ThemeService.toggle()</code> switches <code>themeMode</code> on <code>MaterialApp</code> , updating the entire UI instantly.
Alternative flow (Help & Support)	A dialog displays support email <code>helper.woman.museum@gmail.com</code> , phone 0836 180 180, and hours Monday–Friday 09:00–18:00.
Exception flow	If <code>PATCH /api/profile/update</code> returns an error, the form stays open and a descriptive <code>SnackBar</code> is shown; no input is lost.

Chapter 4

Non-functional Requirements

4.1 Performance

All HTTP requests carry a 30-second timeout to prevent indefinite blocking. Artifact lists are fetched in pages of at most 100 items, preventing excessive data payloads. The museum floor plan (`MapScreen`) is rendered entirely client-side with `CustomPainter`, making it available instantly even without a network connection.

Screen transitions within `LayoutScreen` use `AnimatedSwitcher` (300ms duration). `FutureBuilder` in `RoomScreen` explicitly manages loading, error, and data states, avoiding unnecessary widget rebuilds. Home-screen events are fetched exactly once per session via the `_loaded` flag in `didChangeDependencies`.

4.2 Security

Passwords are never stored or transmitted in plain text. `bcrypt` with salt rounds of 10 guarantees each hash is unique. The TypeORM entity field `passwordHash` carries `select: false`, ensuring it is never included in API responses by default.

The JWT payload contains `{email, sub, isAdmin}`. Every protected request passes through `JwtStrategy` and `JwtAuthGuard`. `AdminGuard` reads `isAdmin` from the decoded payload, eliminating additional database queries per admin action.

A global `ValidationPipe` with `whitelist: true` and `forbidNonWhitelisted: true` strips undeclared DTO fields, preventing mass-assignment vulnerabilities. The `ALLOWED_ORIGINS` environment variable controls CORS; the wildcard default must be restricted before production deployment.

4.3 Usability

The UI is governed by a two-mode theme system. The **Light** theme uses background `#F4EFEB`, text `#16110D`, and primary `#63452C`. The **Dark** theme uses background `#140F0B`, text `#F1EBE7`, and primary `#D4B69D`. Both palettes employ warm brown tones

that complement the museum's historical atmosphere.

The bottom navigation bar uses `BackdropFilter` ($\sigma = 28$) for a glass-morphism effect, rounded corners (radius 48), icon size 34, and label size 14. `BouncingScrollPhysics` is applied consistently across all scrollable lists. `SafeArea` prevents content from being hidden by device status bars or notches.

4.4 Maintainability

Backend modules adhere to the Single Responsibility Principle: each feature (auth, room, artifact, event, profile) has its own controller, service, entity, and DTOs. Field-level validation uses `class-validator` decorators, keeping validation logic separate from business logic.

Frontend responsibilities are cleanly separated across `ApiService` (networking), `AuthService` / `ThemeService` (state), `AppRouter` (routing), and individual screens/widgets (UI). Colors live in `AppColors` and text styles in `AppTextStyles`; a visual rebrand requires editing only two files. Swagger documentation regenerates automatically from NestJS decorators.

4.5 Scalability

The decoupled frontend/backend design allows each tier to scale independently. Supabase provides automatic connection pooling. Pagination ensures API performance is unaffected as the artifact catalogue grows. The NestJS module structure accommodates new features (e.g., QR scanning, audio guide, tour planning) without modifying existing modules.

Chapter 5

External Interface Requirements

5.1 User Interface

LoginScreen. Dark-themed screen with the museum logo at the centre, email and password fields (password supports show/hide), a primary *SIGN IN* button, and a link to **SignupScreen**. The dark background conveys the formal atmosphere of a heritage institution.

LayoutScreen. The application shell. Content occupies the full body area; the glass-morphism bottom bar provides access to four destinations: *Home*, *Map*, *Tour*, and *Profile*.

HomeScreen. A *HeroCard* (230 px tall) with a museum photograph, black gradient overlay, time-contextual greeting (*Good morning/afternoon/evening*), the user's first name in the primary color, and three statistics (visitors today: 500; open rooms: 3; hours: 07:30–17:00) in a dark container. Below: a history card and a scrollable list of **NewsItem** widgets.

MapScreen. Header with title and usage instructions. A styled search bar. Two-floor plan drawn by **CustomPainter**: diamond room shapes, a 10-degree-rotated hallway strip, staircase labels, directional arrows, and a bold *STAIRCASE CONNECTION* divider.

RoomScreen. Header on a semi-transparent primary background showing the floor label and room name with its code. Interactive room map (2px primary border) with numbered artifact dots and inline adjacent-room navigation buttons. Scrollable **ArtifactCard** list with thumbnail, name, and historical date.

ProfileScreen. Circular avatar (initials on primary background), full name, email, and a *Museum member* badge. Personal Information section with an Edit/Save toggle.

Settings section with notification, language, and dark-mode controls. A red-tinted *Sign Out* button at the bottom.

5.2 Software Interfaces

Interface	Specification
Backend REST API	All endpoints prefixed <code>/api/</code> . Default <code>Content-Type: application/json</code> . Auth via <code>Authorization: Bearer {token}</code> . Errors return JSON with a <code>message</code> field.
Supabase PostgreSQL	Connected via <code>DATABASE_URL</code> . SSL required (<code>rejectUnauthorized: false</code>). Connection timeout: 30 s; idle timeout: 10 min.
Swagger UI	Auto-generated at <code>/api/docs</code> . Supports live testing with Bearer auth; <code>persistAuthorization = true</code> retains the token between calls.
SharedPreferences	Key <code>'access_token'</code> : read on startup, cleared on sign-out.

5.3 Communication Interfaces

All client-server traffic uses HTTPS with JSON serialization. The Flutter `http` package enforces a 30-second timeout. NestJS runs on `platform-express`.

CORS permits methods GET, POST, PATCH, DELETE, and OPTIONS; headers `Content-Type` and `Authorization`; origins from `ALLOWED_ORIGINS` (comma-separated). Network errors are surfaced as user-friendly messages; the application does not crash on temporary connectivity loss.

Chapter 6

System Architecture

6.1 Architectural Overview

The system follows a three-layer architecture:

1. **Presentation Layer** – Flutter mobile app: UI rendering and client-side state via Provider.
2. **Business Logic Layer** – NestJS API server: authentication, domain logic, and data access organized as independent modules.
3. **Data Layer** – TypeORM + PostgreSQL on Supabase: all persistent storage.

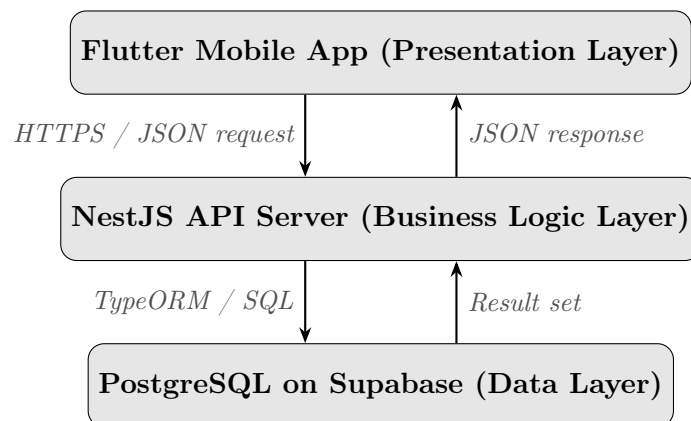


Figure 6.1: Three-layer system architecture

6.2 Backend Module Structure

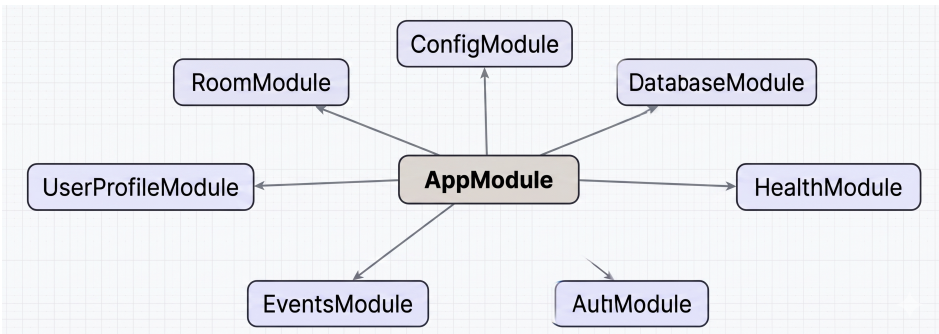


Figure 6.2: System Use Case

6.3 Flutter Screen Hierarchy

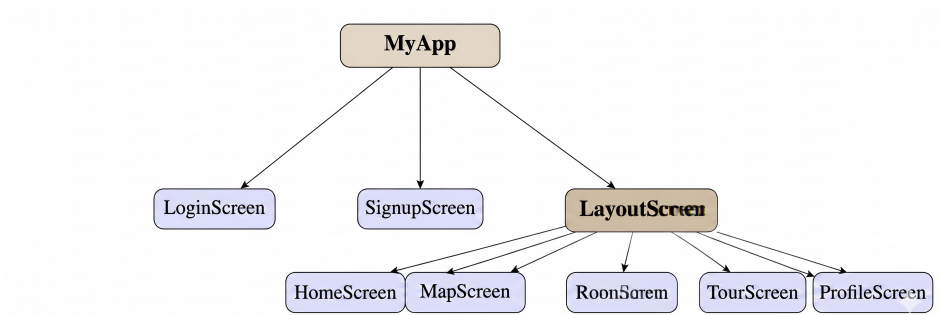


Figure 6.3: System Use Case

6.4 JWT Authentication Data Flow

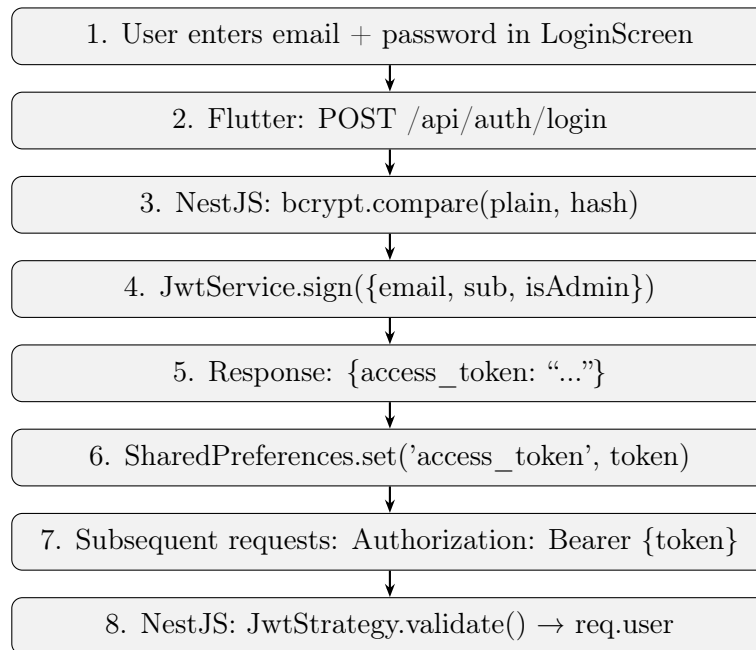


Figure 6.4: JWT authentication data flow

Chapter 7

API Endpoint Summary

Method	Endpoint	Auth	Description
Authentication — /api/auth			
POST	/api/auth/register	None	Register a new user account
POST	/api/auth/login	None	Login; returns JWT access token
Profile — /api/profile			
GET	/api/profile/me	JWT	Get current user's profile
PATCH	/api/profile/update	JWT	Update user profile fields
GET	/api/profile/:id	JWT (self/admin)	Get user by ID
Rooms — /api/rooms			
POST	/api/rooms/all	None	Paginated room list
GET	/api/rooms?code=	None	Get room by code
POST	/api/rooms	JWT + Admin	Create a new room
PATCH	/api/rooms/:id	JWT + Admin	Update a room
DELETE	/api/rooms	JWT + Admin	Delete rooms by IDs
Artifacts — /api/artifacts			
POST	/api/artifacts/all	None	Paginated artifact list with filter
GET	/api/artifacts	None	Get artifact by roomId + orderNo
POST	/api/artifacts	JWT + Admin	Create a new artifact
PATCH	/api/artifacts/:id	JWT + Admin	Update an artifact
PATCH	/api/artifacts/:id/position	JWT + Admin	Update artifact map position

Method	Endpoint	Auth	Description
DELETE	/api/artifacts	JWT + Admin	Delete artifacts by IDs
Events — /api/events			
GET	/api/events	None	List all events
GET	/api/events/:id	None	Get event by ID
POST	/api/events	JWT + Admin	Create a new event
PATCH	/api/events/:id	JWT + Admin	Update an event
DELETE	/api/events/:id	JWT + Admin	Delete an event
Health			
GET	/health	None	Server health check

Chapter 8

Database Schema Summary

Table: users

Column	Type	Notes
id	UUID	Primary key
firstName, lastName	varchar	
email	varchar	Unique
phone	varchar	
passwordHash	varchar	Not selected by default
isAdmin	bool	Default: false
language	varchar	Default: 'en'
isNotificationEnabled	bool	Default: true
createdAt, updatedAt	timestampz	Auto-managed

Table: rooms

Column	Type	Notes
id	UUID	Primary key
name	varchar	Vietnamese name
nameEn	varchar	English name
code	varchar	Unique; e.g., R1.1, R2.2, R3
description	varchar	Vietnamese description
descriptionEn	varchar	English description
createdAt, updatedAt	timestampz	Auto-managed

Table: artifacts

Column	Type	Notes
id	UUID	Primary key

Column	Type	Notes
roomId	UUID	FK → rooms (CASCADE)
name	varchar	Vietnamese name
description	varchar	Vietnamese; nullable
descriptionEn	varchar	English; nullable
orderNo	bigint	Display order within room
imgUrl	varchar	Nullable
historyDate	timestampz	Nullable
positionX	double precision	Normalised 0.0–1.0
positionY	double precision	Normalised 0.0–1.0
createdAt, updatedAt	timestampz	Auto-managed

Table: events

Column	Type	Notes
id	UUID	Primary key
title	varchar	
description	text	
date	varchar	
imageUrl	varchar	Nullable
status	enum	active / inactive / ended; default: active
createdAt, updatedAt	timestampz	Auto-managed